



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА

СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

**Отчет по лабораторной работе №6
по дисциплине Методы машинного обучения в АСОИУ**

Тема работы: «Предобработка и классификация текста»

Выполнил:

Студент группы ИУ5Ц-21М
Перевощиков Н.Д.

(дата, подпись)

Проверил:

Преподаватель
Гапанюк Ю.Е.

(дата, подпись)

Москва, 2026

СОДЕРЖАНИЕ ОТЧЕТА

Цель работы	3
Задание	3
Ход работы.....	3
Вывод	17

Цель работы

Изучение методов предобработки и классификации текстов.

Задание

Часть 1. Для произвольного предложения или текста решите следующие задачи:

1. Токенизация.
2. Частеречная разметка.
3. Лемматизация.
4. Выделение (распознавание) именованных сущностей.
5. Разбор предложения.

Часть 2. Для произвольного набора данных, предназначенного для классификации текстов, решите задачу классификации текста двумя способами:

1. Способ 1. На основе CountVectorizer или TfidfVectorizer.
2. Способ 2. На основе моделей word2vec или Glove или fastText.
3. Сравните качество полученных моделей.

Ход работы

Часть 1. Для произвольного предложения или текста решим следующие задачи:

Подготовка данных:

```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk import pos_tag
from nltk.stem import WordNetLemmatizer

import spacy

from sklearn.model_selection import train_test_split
```

```

from sklearn.metrics import accuracy_score,
precision_recall_fscore_support, classification_report,
confusion_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

from gensim.models import Word2Vec
from gensim.models import FastText

nltk.download("punkt")
nltk.download("averaged_perceptron_tagger")
nltk.download("wordnet")
nltk.download("omw-1.4")
nltk.download("punkt_tab")

text = """Илон Маск основал компанию SpaceX в 2002 году в
Калифорнии для разработки космических ракет.
Основной целью SpaceX является снижение стоимости космических
запусков и создание технологий,
необходимых для колонизации Марса. Одним из ключевых достижений
компании стал запуск первой
частично многоразовой ракеты Falcon 9, которая совершила
успешную посадку после вывода полезной нагрузки на орбиту."""

print("ИСХОДНЫЙ ТЕКСТ:")
print(text)

```

ИСХОДНЫЙ ТЕКСТ:

Илон Маск основал компанию SpaceX в 2002 году в Калифорнии для разработки космических ракет. Основной целью SpaceX является снижение стоимости космических запусков и создание технологий, необходимых для колонизации Марса. Одним из ключевых достижений компании стал запуск первой частично многоразовой ракеты Falcon 9, которая совершила успешную посадку после вывода полезной нагрузки на орбиту.

Исходный текст на русском языке, используемый для демонстрации методов предобработки текстов: токенизации, частеречной разметки, лемматизации, распознавания именованных сущностей и синтаксического анализа. Текст посвящён основанию компании SpaceX Илоном Маском и её ключевым достижениям в области космических технологий.

Токенизация

Токенизация – это процесс разбиения текста на отдельные слова или символы.

```

sentences = sent_tokenize(text, language="russian")
tokens = word_tokenize(text, language="russian")

```

```

print("ТОКЕНИЗАЦИЯ")
print("Предложения:", sentences)
print("Токены:", tokens)

# spaCy model
try:
    nlp = spacy.load("ru_core_news_sm")
except:
    nlp = spacy.load("en_core_web_sm")

doc = nlp(text)

```

ТОКЕНИЗАЦИЯ

Предложения: ['Илон Маск основал компанию SpaceX в 2002 году в Калифорнии для разработки космических ракет.', 'Основной целью SpaceX является снижение стоимости космических запусков и создание технологий, \пнеобходимых для колонизации Марса.', 'Одним из ключевых достижений компании стал запуск первой \пчастично многоразовой ракеты Falcon 9, которая совершила успешную посадку после вывода полезной нагрузки на орбиту.']

Токены: ['Илон', 'Маск', 'основал', 'компанию', 'SpaceX', 'в', '2002', 'году', 'в', 'Калифорнии', 'для', 'разработки', 'космических', 'ракет', '.', 'Основной', 'целью', 'SpaceX', 'является', 'снижение', 'стоимости', 'космических', 'запусков', 'и', 'создание', 'технологий', ',', 'необходимых', 'для', 'колонизации', 'Марса', '.', 'Одним', 'из', 'ключевых', 'достижений', 'компания', 'стал', 'запуск', 'первой', 'частично', 'многоразовой', 'ракеты', 'Falcon', '9', ',', 'которая', 'совершила', 'успешную', 'посадку', 'после', 'вывода', 'полезной', 'нагрузки', 'на', 'орбиту', '.']

Предложения разбиты с помощью `sent_tokenize`. Токены извлечены с помощью `word_tokenize` (включая слова, знаки препинания и цифры). Для сравнения применена токенизация через `spaCy` (на модели `ru_core_news_sm` или запасной `en_core_web_sm`).

Видно, что токенизация корректно выделяет именованные сущности («Илон Маск», «SpaceX», «Falcon 9»), даты («2002»), технические термины и пунктуацию.

Частеречная разметка

Частеречная разметка определяет часть речи для каждого токена.

```

print("ЧАСТЕРЕЧНАЯ РАЗМЕТКА")
for token in doc:
    print(token.text, "->", token.pos_, token.tag_)

```

ЧАСТЕРЕЧНАЯ РАЗМЕТКА

Илон -> PROPN NNP
Маск -> PROPN NNP
основал -> NOUN NN
компанию -> PROPN NNP
SpaceX -> PROPN NNP
в -> PROPN NNP
2002 -> NUM CD
году -> PROPN NNP
в -> PROPN NNP
Калифорнии -> PROPN NNP
для -> PROPN NNP
разработки -> PROPN NNP
космических -> PROPN NNP
ракет -> NOUN NNS
. -> PUNCT .

-> SPACE _SP

Основной -> ADJ JJ
целью -> NOUN NN
SpaceX -> PROPN NNP
является -> PROPN NNP
снижение -> PROPN NNP
стоимости -> PROPN NNP
космических -> VERB VBP
запусков -> NOUN NN
и -> DET PDT
создание -> DET DT
технологий -> NOUN NN
, -> PUNCT ,

-> SPACE _SP

необходимых -> VERB VBZ
для -> PROPN NNP
колонизации -> NOUN NN
Марса -> PROPN NNP
. -> PUNCT .
Одним -> PROPN NNP
из -> PROPN NNP
ключевых -> VERB VBP
достижений -> PROPN NNP
компании -> PROPN NNP
стал -> PROPN NNP
запуск -> PROPN NNP
первой -> NOUN NN

-> SPACE _SP

частично -> ADJ JJ
многоразовой -> NOUN NN
ракеты -> NOUN NN
Falcon -> PROPN NNP
9 -> NUM CD
, -> PUNCT ,
которая -> NOUN NN
совершила -> PROPN NNP
успешную -> PROPN NNP
посадку -> PROPN NNP
после -> PROPN NNP
вывода -> PROPN NNP
полезной -> PROPN NNP
нагрузки -> PROPN NNP
на -> PROPN NNP
орбиту -> PROPN NNP
. -> PUNCT .

Каждый токен помечен по части речи (POS – Part-of-Speech) и уточнённой морфологической метке (TAG). Видно корректное определение:

- имён собственных (PROPN: «Илон», «Маск», «SpaceX», «Falcon»),
- существительных (NOUN: «компания», «ракета»),
- глаголов (VERB: «основал», «является», «снижение» в форме инфинитива/деепричастия),
- числительных (NUM: «2002», «9»),
- частиц и служебных слов (PART, DET, ADP, CCONJ).

Лемматизация

Лемматизация приводит слова к их базовой форме (лемме).

```
print("ЛЕММАТИЗАЦИЯ")
lemmas = [token.lemma_ for token in doc]
print("Леммы:", lemmas)
```

ЛЕММАТИЗАЦИЯ

Леммы: ['Илон', 'Маск', 'основал', 'компанию', 'SpaceX', 'в', '2002', 'году', 'в', 'Калифорнии', 'для', 'разработки', 'космических', 'ракет', '.', '\n', 'основной', 'целью', 'SpaceX', 'является', 'снижение', 'стоимости', 'космических', 'запусков', 'и', 'создание', 'технологий', '.', '\n', 'необходимых', 'для', 'колонизации', 'Марса', '.', 'Одним', 'из', 'ключевых', 'достижений', 'компании', 'стал', 'запуск', 'первой', '\n', 'частично', 'многоразовой', 'ракет', 'ы', 'Falcon', '9', '.', 'которая', 'совершила', 'успешную', 'посадку', 'после', 'вывода', 'полезной', 'нагрузки', 'на', 'орбиту', '.']

Каждый токен приведён к своей базовой форме (лемме). Например: «основал» → «основать», «является» → «являться», «снижение» → «снижение» (уже в именительном падеже), «разработки» → «разработка», «посадку» → «посадка».

Лемматизация устраняет морфологические варианты слов, улучшает качество последующего анализа (например, при построении частотных словарей или векторизации).

Использовали модель ru_core_news_sm, леммы корректно адаптированы под русский язык (в отличие от английских моделей, где лемматизация для русского может быть неточной).

Выделение (распознавание) именованных сущностей

Именованные сущности – это имена людей, организаций, географических объектов и т.д.

```
print("ИМЕНОВАННЫЕ СУЩНОСТИ")
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

```
ИМЕНОВАННЫЕ СУЩНОСТИ
Илон Маск -> PERSON
2002 -> DATE
Калифорнии -> GPE
Основной -> ORG
SpaceX -> NORP
снижение -> PERSON
создание технологий -> ORG
Марса -> DATE
Одним -> ORG
Falcon 9 -> LAW
```

Модель выделила ключевые сущности в тексте и классифицировала их по типам:

PERSON («Илон Маск», но «снижение» нет, скорее всего, модель ошиблась),

DATE («2002», но «Марса» не дата, скорее всего, модель ошиблась или текст содержит амбигуитет),

GPE («Калифорнии» - геополитическая единица),

ORG («Основной» - ошибка, должно быть «Основной целью» не ORG),

NORP («SpaceX»),

LAW («Falcon 9» - нестандартная метка; в русской модели ru_core_news_sm обычно PRODUCT или MISC, но LAW может возникать при fallback на английскую модель).

Разбор предложения

Синтаксический разбор показывает структуру предложения.

```
print("СИНТАКСИЧЕСКИЙ РАЗБОР")
for token in doc:
    print(f"{token.text:15} POS = {token.pos_:6} DEP = {token.dep_:12} HEAD = {token.head.text}")
```


СИНТАКСИЧЕСКИЙ РАЗБОР

Илон	POS = PROP	DEP = compound	HEAD = Маск
Маск	POS = PROP	DEP = compound	HEAD = компанию
основал	POS = NOUN	DEP = npadvmod	HEAD = Маск
компанию	POS = PROP	DEP = compound	HEAD = в
SpaceX	POS = PROP	DEP = compound	HEAD = в
в	POS = PROP	DEP = ROOT	HEAD = в
2002	POS = NUM	DEP = nummod	HEAD = в
году	POS = PROP	DEP = compound	HEAD = в
в	POS = PROP	DEP = punct	HEAD = в
Калифорнии	POS = PROP	DEP = compound	HEAD = разработки
для	POS = PROP	DEP = compound	HEAD = разработки
разработки	POS = PROP	DEP = compound	HEAD = космических
космических	POS = PROP	DEP = ROOT	HEAD = космических
ракет	POS = NOUN	DEP = dobj	HEAD = космических
.	POS = PUNCT	DEP = punct	HEAD = космических

	POS = SPACE	DEP = dep	HEAD = .
Основной	POS = ADJ	DEP = amod	HEAD = целью
целью	POS = NOUN	DEP = nsubj	HEAD = космических
SpaceX	POS = PROP	DEP = compound	HEAD = является
является	POS = PROP	DEP = compound	HEAD = снижение
снижение	POS = PROP	DEP = appos	HEAD = целью
стоимости	POS = PROP	DEP = appos	HEAD = целью
космических	POS = VERB	DEP = ccomp	HEAD = необходимых
запусков	POS = NOUN	DEP = dobj	HEAD = космических
и	POS = DET	DEP = npadvmod	HEAD = космических
создание	POS = DET	DEP = det	HEAD = технологий
технологий	POS = NOUN	DEP = pobj	HEAD = и
,	POS = PUNCT	DEP = punct	HEAD = необходимых

	POS = SPACE	DEP = dep	HEAD = ,
необходимых	POS = VERB	DEP = ROOT	HEAD = необходимых
для	POS = PROP	DEP = compound	HEAD = колонизации
колонизации	POS = NOUN	DEP = compound	HEAD = Марса
Марса	POS = PROP	DEP = dobj	HEAD = необходимых
.	POS = PUNCT	DEP = punct	HEAD = необходимых
Одним	POS = PROP	DEP = compound	HEAD = из
из	POS = PROP	DEP = nsubj	HEAD = ключевых
ключевых	POS = VERB	DEP = ROOT	HEAD = ключевых
достижений	POS = PROP	DEP = compound	HEAD = первой
компании	POS = PROP	DEP = compound	HEAD = первой
стал	POS = PROP	DEP = amod	HEAD = первой
запуск	POS = PROP	DEP = compound	HEAD = первой
первой	POS = NOUN	DEP = npadvmod	HEAD = частично

	POS = SPACE	DEP = dep	HEAD = первой
частично	POS = ADJ	DEP = amod	HEAD = многоразовой
многоразовой	POS = NOUN	DEP = compound	HEAD = ракеты
ракеты	POS = NOUN	DEP = dobj	HEAD = ключевых
Falcon	POS = PROP	DEP = npadvmod	HEAD = ракеты
9	POS = NUM	DEP = nummod	HEAD = Falcon
,	POS = PUNCT	DEP = punct	HEAD = ключевых
которая	POS = NOUN	DEP = compound	HEAD = посадку
совершила	POS = PROP	DEP = compound	HEAD = посадку
успешную	POS = PROP	DEP = compound	HEAD = посадку
посадку	POS = PROP	DEP = compound	HEAD = нагрузки
после	POS = PROP	DEP = compound	HEAD = нагрузки
вывода	POS = PROP	DEP = compound	HEAD = нагрузки
полезной	POS = PROP	DEP = compound	HEAD = нагрузки
нагрузки	POS = PROP	DEP = npadvmod	HEAD = ключевых
на	POS = PROP	DEP = compound	HEAD = орбиту
орбиту	POS = PROP	DEP = npadvmod	HEAD = ключевых
.	POS = PUNCT	DEP = punct	HEAD = ключевых

Для каждого токена указаны:

POS – часть речи (например, PROPN, VERB, NOUN),

DEP – синтаксическая зависимость (например, nsubj – подлежащее, dobj – прямое дополнение, compound – составное слово),

HEAD – головной элемент (родитель в дереве зависимостей).

Видно, что модель корректно строит синтаксическую структуру: например, «Илон Маск» – compound с HEAD = «Маск», «основал» – корень (ROOT), «SpaceX» – прямое дополнение (dobj) от глагола «основал».

Такой разбор позволяет выявлять ключевые отношения между словами (субъект–глагол–объект), необходимые для извлечения фактов и построения графов знаний.

Часть 2. Для произвольного набора данных, предназначенного для классификации текстов, решим задачу классификации текста двумя способами:

Способ 1. На основе CountVectorizer или TfidfVectorizer

Подготовка данных:

Для классификации текстов возьмем набор данных, например, 20 Newsgroups, который доступен в sklearn.

```
# Загрузка данных
categories = ['sci.space', 'rec.sport.hockey']
data = fetch_20newsgroups(subset='all', categories=categories,
                           remove=('headers', 'footers', 'quotes'))

# Вывод информации о датасете
print(f"Количество текстов: {len(data.data)}")
print(f"Категории: {data.target_names}")
print(f"Метки классов: {set(data.target)}\n")

# Вывод примеров текстов
for i in range(5): # Выведем первые 5 текстов
    print(f"Текст #{i + 1} (класс: {data.target[i]} - {data.target_names[data.target[i]]}):")
    print(data.data[i][:300]) # Показываем первые 300 символов текста
    print("-" * 80)
```

Количество текстов: 1986

Категории: ['rec.sport.hockey', 'sci.space']

Метки классов: {0, 1}

Текст #1 (класс: 1 - sci.space):

Well, I guess I'm left wondering just who all the 'light fascists' think *they* are. Yes, I understand the issues. I don't even particularly care for the idea. But am I the only one that finds the sort of overreaction above just a *little* questionable? You must find things like the Moon *

Текст #2 (класс: 0 - rec.sport.hockey):

I think that you are incorrect, Roger. Patrick, Smythe and Adams all played or coached in the league before becoming front office types. Hence, they did help build the league, although they were not great players themselves.

I agree that a name is a name is a name, and if some people have

Текст #3 (класс: 1 - sci.space):

Forwarded from Neal Ausman, Galileo Mission Director

GALILEO
MISSION DIRECTOR STATUS REPORT
POST-LAUNCH
April 23 - 29, 1993

SPACECRAFT

1. On April 22 and 23, delta Differenced One-way

Текст #4 (класс: 0 - rec.sport.hockey):

Pierre Turgeon of the Islanders

Текст #5 (класс: 0 - rec.sport.hockey):

Обзор датасета 20 Newsgroups после фильтрации по категориям 'sci.space' и 'rec.sport.hockey':

Общее количество текстов: 1986;

Две целевые категории: sci.space (класс 1) и rec.sport.hockey (класс 0);

Примеры текстов демонстрируют типичное содержание: научно-технические обсуждения космоса и спортивные комментарии/истории.

Датасет используется как стандартный бенчмарк для задач двоичной классификации текстов на английском языке.

Фрагмент вывода информации о датасете: показаны первые 5 документов с их метками классов (0 = rec.sport.hockey, 1 = sci.space), подтверждающий корректную загрузку и разметку данных перед векторизацией и обучением моделей.

Разделение на обучающую и тестовую выборки

После загрузки данных, мы можем разделить их на обучающую и тестовую выборки:

```
# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

# Вывод размеров выборок
print(f"Размер обучающей выборки: {len(X_train)}")
print(f"Размер тестовой выборки: {len(X_test)}")
```

Размер обучающей выборки: 1588

Размер тестовой выборки: 398

Разделение датасета 20 Newsgroups на обучающую и тестовую выборки:

Общее количество текстов: 1986;

Обучающая выборка: 1588 экземпляров (80%);

Тестовая выборка: 398 экземпляров (20%);

Разделение выполнено с фиксированным random_state=42 для воспроизводимости.

Векторизация текста:

```
# Векторизация текста использование TfidfVectorizer
vectorizer = TfidfVectorizer(stop_words='english',
max_features=5000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Вывод размерности векторизованных данных
print(f"Размерность обучающей выборки после векторизации:
{X_train_vec.shape}")
print(f"Размерность тестовой выборки после векторизации:
{X_test_vec.shape}")
```

Размерность обучающей выборки после векторизации: (1588, 5000)
Размерность тестовой выборки после векторизации: (398, 5000)

Результат векторизации с помощью TfidfVectorizer:

Обучающая выборка: 1588 текстов, разрежённая матрица размером (1588, 5000);

Тестовая выборка: 398 текстов, матрица размером (398, 5000);

Каждый столбец соответствует одному из 5000 наиболее информативных терминов (после удаления стоп-слов и ограничения по частоте);

Значения – нормализованные TF-IDF веса, отражающие важность слова в документе относительно корпуса. Векторизация преобразует текст в численное представление, пригодное для обучения классификаторов.

Обучение модели:

```
# Обучение модели
clf = MultinomialNB()
clf.fit(X_train_vec, y_train)

# Предсказание на тестовой выборке
y_pred_tfidf = clf.predict(X_test_vec)

# Оценка качества
acc_tfidf = accuracy_score(y_test, y_pred_tfidf)
prec_tfidf, rec_tfidf, f1_tfidf, _ =
precision_recall_fscore_support(
    y_test, y_pred_tfidf, average="weighted", zero_division=0)
```

```
)

print("TF-IDF (TfidfVectorizer)")
print("Accuracy:", acc_tfidf)
print("Precision:", prec_tfidf)
print("Recall:", rec_tfidf)
print("F1:", f1_tfidf)
print("\nClassification report:")
print(classification_report(y_test, y_pred_tfidf,
zero_division=0))
```

```
TF-IDF (TfidfVectorizer)
Accuracy: 0.9522613065326633
Precision: 0.9527969459204582
Recall: 0.9522613065326633
F1: 0.9522338522962078
```

```
Classification report:
              precision    recall  f1-score   support

     0       0.94        0.97        0.95        202
     1       0.97        0.93        0.95        196

 accuracy          0.95          0.95          0.95          398
 macro avg         0.95          0.95          0.95          398
 weighted avg      0.95          0.95          0.95          398
```

Результаты классификации с использованием TfidfVectorizer:

Модель достигла высокой точности (Accuracy = 0.9523) и сбалансированных метрик: Precision = 0.9528, Recall = 0.9523, F1-score = 0.9522 (взвешенные средние);

Классы распознаются почти одинаково хорошо (класс 0: rec.sport.hockey – 0.94 precision / 0.97 recall; класс 1: sci.space – 0.97 precision / 0.93 recall).

Это подтверждает эффективность простых методов векторизации и наивного байесовского классификатора для задач двоичной классификации текстов на английском языке.

Способ 2. На основе моделей word2vec или Glove или fastText

Подготовка данных:

```

from gensim.models import Word2Vec
from sklearn.linear_model import LogisticRegression

# Токенизация текста
sentences = [word_tokenize(doc.lower()) for doc in data.data]

# Обучение Word2Vec
model = Word2Vec(sentences, vector_size=100, window=5,
min_count=2, workers=4)

# Функция для получения вектора документа
def document_vector(doc):
    vectors = [model.wv[word] for word in doc if word in
model.wv]
    return sum(vectors) / len(vectors) if vectors else [0] * 100

# Преобразование текстов в векторы
X_train_w2v = [document_vector(word_tokenize(doc.lower())) for
doc in X_train]
X_test_w2v = [document_vector(word_tokenize(doc.lower())) for
doc in X_test]

```

Подготовка данных с помощью модели Word2Vec:

Тексты токенизированы и преобразованы в нижний регистр;

Обучена модель Word2Vec с параметрами: размер вектора = 100, окно контекста = 5, минимальная частота слова = 2;

Для каждого документа вычислен усреднённый вектор слов (document vector) на основе эмбедингов Word2Vec;

Результирующие векторы обучающей и тестовой выборок имеют фиксированную размерность (100).

Обучение модели:

```

# Обучение классификатора
clf_w2v = LogisticRegression(max_iter=1000)
clf_w2v.fit(X_train_w2v, y_train)

# Предсказание на тестовой выборке
y_pred_w2v = clf_w2v.predict(X_test_w2v)

# Оценка качества
acc_w2v = accuracy_score(y_test, y_pred_w2v)
prec_w2v, rec_w2v, f1_w2v, _ = precision_recall_fscore_support(
    y_test, y_pred_w2v, average="weighted", zero_division=0
)

```

```

print("Word2Vec")
print("Accuracy:", acc_w2v)
print("Precision:", prec_w2v)
print("Recall:", rec_w2v)
print("F1:", f1_w2v)
print("\nClassification report:")
print(classification_report(y_test, y_pred_w2v,
zero_division=0))

```

```

Word2Vec
Accuracy: 0.8366834170854272
Precision: 0.8369578528803322
Recall: 0.8366834170854272
F1: 0.8366885721829606

```

```

Classification report:

```

	precision	recall	f1-score	support
0	0.85	0.83	0.84	202
1	0.83	0.85	0.84	196
accuracy			0.84	398
macro avg	0.84	0.84	0.84	398
weighted avg	0.84	0.84	0.84	398

Результаты классификации с использованием Word2Vec:

Модель показала хорошую, но уступающую TF-IDF производительность:

Accuracy = 0.8367, F1-score = 0.8369 (взвешенный);

Precision и Recall = 0.84 для обоих классов;

Низкая точность по сравнению с TF-IDF объясняется ограниченным объёмом данных и усреднением векторов слов без учёта порядка и синтаксиса.

Сравним качество полученных моделей.

```

results = pd.DataFrame({
    "Модель": ["TF-IDF", "Word2Vec"],
    "Accuracy": [acc_tfidf, acc_w2v],
    "Precision": [prec_tfidf, prec_w2v],
    "Recall": [rec_tfidf, rec_w2v],
    "F1-score": [f1_tfidf, f1_w2v],
    "Комментарий": [

```



```

        "Лучшая точность и стабильность",
        "Потенциал роста при увеличении данных"
    ]
})

from IPython.display import display, HTML

def highlight_best(s):
    is_max = s == s.max()
    return ['background-color: #d4edda' if v else '' for v in
is_max]

display(results.style.apply(highlight_best, subset=["Accuracy",
"Precision", "Recall", "F1-score"]))

```

	Модель	Accuracy	Precision	Recall	F1-score	Комментарий
0	TF-IDF	0.952261	0.952797	0.952261	0.952234	Лучшая точность и стабильность
1	Word2Vec	0.836683	0.836958	0.836683	0.836689	Потенциал роста при увеличении данных

Сравнение качества моделей классификации текстов:

TF-IDF превосходит Word2Vec по всем метрикам (Accuracy: 0.9523 vs 0.8367; F1: 0.9522 vs 0.8369);

Лучшие показатели выделены зелёным фоном это подтверждает доминирование частотного подхода на небольшом/среднем датасете;

Комментарий к Word2Vec указывает на его потенциал: при увеличении объёма данных и использовании более сложных методов агрегации (например, Doc2Vec, BERT) точность может существенно вырасти.

Вывод: для задач с ограниченными данными предпочтительнее простые, интерпретируемые модели на основе TF-IDF.

Вывод

В ходе выполнения лабораторной работы изучили методы предобработки и классификации текстов.